EECE5644 Fall 2026 — Assignment 1

David Abu

October 2026

Declaration of external resources and AI use. Claude (Anthropic) generated the initial implementation and report. OpenAI Codex reviewed and reran the experiments, corrected ROC integration and validation preprocessing, added official-data provenance and numerical checks, and refined this report. The mathematical derivations and code are AI-assisted; the independent reruns and numerical checks are described below. Software: Python, NumPy, SciPy, pandas, Matplotlib, and LaTeX. The experiments reported here use the original UCI Wine Quality and Human Activity Recognition archives; exact download URLs and SHA-256 hashes are included below. The instructor's assignment permits AI assistance when it is acknowledged.

Notation and loss convention. λ_{ij} is the loss for deciding $D = i$ when the true label is $L = j$. All random data are generated once with fixed seeds, saved to disk, and the same data are used for every part of a question.

Question 1: Gaussian classifiers and Fisher LDA

Generated $N = 10{,}000$ samples ($L = 1$ with probability 0.65): $N_0 = 3460$, $N_1 = 6540$. Seed 5644; data saved in `q1_data.npz`.

Part A: ERM with the true pdf

1. Decision rule. The conditional risk of decision i is $R(D{=}i \mid \mathbf{x}) = \sum_j \lambda_{ij} P(L{=}j \mid \mathbf{x})$. Deciding $D = 1$ is optimal iff $R(D{=}1|\mathbf{x}) < R(D{=}0|\mathbf{x})$:

$$\begin{aligned} &\lambda_{10} P(L{=}0|\mathbf{x}) + \lambda_{11} P(L{=}1|\mathbf{x}) < \lambda_{00} P(L{=}0|\mathbf{x}) + \lambda_{01} P(L{=}1|\mathbf{x}) \\ \iff &\left(\lambda_{01} - \lambda_{11}\right) p(\mathbf{x}|L{=}1) P(L{=}1) > \left(\lambda_{10} - \lambda_{00}\right) p(\mathbf{x}|L{=}0) P(L{=}0), \end{aligned}$$

using Bayes' rule and cancelling $p(\mathbf{x})$. Assuming $\lambda_{01} > \lambda_{11}$, the minimum-expected-risk rule is the likelihood-ratio test

$$\boxed{\frac{p(\mathbf{x} \mid L=1)}{p(\mathbf{x} \mid L=0)} \underset{D=0}{\overset{D=1}{\gtrless}} \gamma = \frac{\left(\lambda_{10} - \lambda_{00}\right) P(L=0)}{\left(\lambda_{01} - \lambda_{11}\right) P(L=1)}}$$

For arbitrary nonnegative losses, the undivided inequality above remains valid. If $\lambda_{01} - \lambda_{11} < 0$, division reverses the inequality; if it is zero, compare the remaining term directly. Equal conditional risks may be resolved by a fixed tie convention.

For minimum probability of error (0–1 loss: $\lambda_{00} = \lambda_{11} = 0$, $\lambda_{01} = \lambda_{10} = 1$): $\gamma^* = P(L{=}0)/P(L{=}1) = 0.35/0.65 = 0.5385$, $\ln \gamma^* = -0.619$. With Gaussian class conditionals the test is implemented in the log domain (numerically stable, monotone so equivalent):

$$\ln \frac{p(\mathbf{x}|1)}{p(\mathbf{x}|0)} = -\tfrac{1}{2}(\mathbf{x} - \mathbf{m}_1)^T C_1^{-1} (\mathbf{x} - \mathbf{m}_1) + \tfrac{1}{2}(\mathbf{x} - \mathbf{m}_0)^T C_0^{-1} (\mathbf{x} - \mathbf{m}_0) - \tfrac{1}{2} \ln \frac{|C_1|}{|C_0|} \;\gtrless\; \ln \gamma.$$

This is a quadratic discriminant, since $C_0 \neq C_1$.

2. ROC. The score $s(\mathbf{x}_n) = \ln[p(\mathbf{x}_n|1)/p(\mathbf{x}_n|0)]$ was computed for all samples. Sweeping $\ln \gamma$ from $-\infty$ through every observed score value (10,001 thresholds) gives every distinct operating point the data can produce; for each, $P(D{=}1|L{=}1;\gamma)$ and $P(D{=}1|L{=}0;\gamma)$ are estimated by counts. At $\gamma = 0$ the curve is at $(1, 1)$; at $\ln \gamma = \max_n s(\mathbf{x}_n)$

(finite) it reaches $(0, 0)$. Both $P(D{=}1|L{=}0;\gamma)$ and $P(D{=}0|L{=}1;\gamma) = 1 - P(D{=}1|L{=}1;\gamma)$ are stored for each threshold.

3. Minimum-P(error) threshold. For each threshold,

$$P(\text{error};\gamma) = P(D{=}1|L{=}0;\gamma)P(L{=}0) + P(D{=}0|L{=}1;\gamma)P(L{=}1);$$

the minimizer is marked on the ROC (Fig. 1). Any γ between two consecutive sorted scores gives the same decisions; I report the midpoint. Results are in Table 1.

Classifier	threshold at min $P(e)$	$P(D{=}1\|L{=}1)$	$P(D{=}1\|L{=}0)$	$\hat{P}$(error)	AUC
A: true pdf	$\hat{\gamma} = 0.791$ $(\ln\hat{\gamma} = -0.235)$	0.9679	0.0145	0.0259	0.9959
at theory $\gamma^* = 0.5385$	$\ln\gamma^* = -0.619$	0.9716	0.0225	0.0264	
B: naive Bayes	$\hat{\gamma} = 0.640$ $(\ln\hat{\gamma} = -0.447)$	0.9654	0.0286	0.0325	0.9942
at $\gamma^* = 0.5385$	$\ln\gamma^* = -0.619$	0.9670	0.0318	0.0326	
C: Fisher LDA	$\hat{\tau} = -0.0053$	0.9512	0.0584	0.0521	0.9854

Table 1: Question 1 summary. Probabilities are sample-count estimates on the same 10K samples.

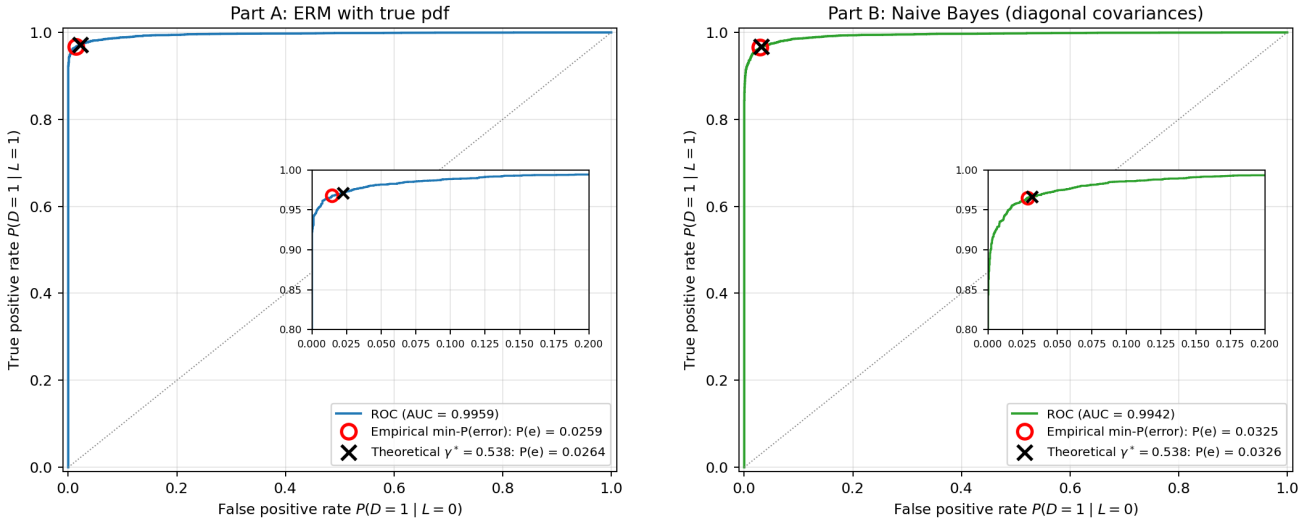


Figure 1: ROC curves of the ERM classifier with the true pdf (left) and the naive Bayes classifier (right). Red circle: empirically selected minimum-P(error) operating point; black cross: operating point at the theoretical threshold $\gamma^* = 0.5385$. Insets zoom into the upper-left corner.

Estimated minimum P(error) $\approx$ 0.0259 (at $\hat{\gamma} \approx 0.79$), versus 0.0264 at the theoretical $\gamma^* = 0.538$. As an independent reference, I evaluated the theoretical rule on a separate 2-million-sample Monte Carlo set: $P(\text{error};\gamma^*) \approx 0.0262$, with binomial Monte Carlo standard error approximately 0.000113 (an approximate 95% interval is 0.0262235 ± 0.0002215). This estimates the Bayes error; it is not an exact analytic value.

Comparison of empirical vs. theoretical γ. The empirical minimizer ($\hat{\gamma} \in [0.783, 0.799)$) is larger than $\gamma^* = 0.538$, yet the two error values differ by only 0.00045 (about 4–5 of 10,000 samples). Figure 2 shows why: $P(\text{error};\gamma)$ is very flat around its minimum, so with a finite sample the location of the minimum is noisy. The empirical $\hat{\gamma}$ is fitted to these particular 10K samples, so its error 0.0259 is optimistically biased (below the large-sample Bayes error 0.0262). The theoretically derived γ^* is the one that is optimal for the distribution; the empirical one only agrees with it up to sampling noise, and would converge to it as $N \to \infty$.

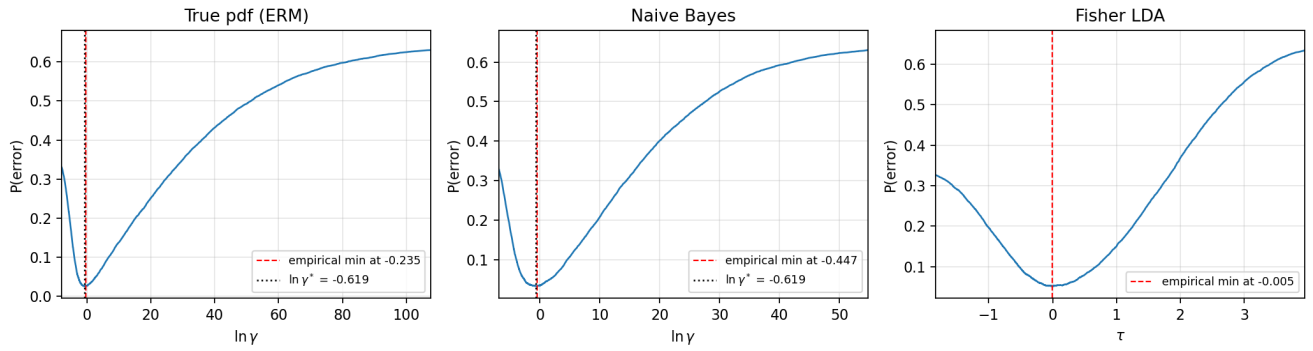


Figure 2: Estimated $P(\text{error})$ versus threshold for the three classifiers. Red dashed: empirical minimizer; black dotted: $\ln \gamma^*$.

Part B: Naive Bayes (diagonal covariance) model mismatch

The classifier uses the same LRT and threshold formula but with $C_0^{NB} = \text{diag}(C_0) = \text{diag}(0.5, 0.25, 0.25, 0.5)$ and $C_1^{NB} = \text{diag}(C_1) = \text{diag}(1, 2, 1, 3)$, i.e. it ignores the correlations among x_1, x_2, x_3 (x_4 is already uncorrelated). The score is $\ln g(\mathbf{x}|\mathbf{m}_1, C_1^{NB}) - \ln g(\mathbf{x}|\mathbf{m}_0, C_0^{NB})$ and the same sweep is performed on the same 10K samples (Fig. 1 right, Table 1).

- Minimum $\hat{P}(\text{error}) = \mathbf{0.0325}$ at $\hat{\gamma} \approx 0.640$ (vs. 0.0326 at $\gamma^* = 0.538$). Monte Carlo with 2M samples at γ^*: 0.0322.
- The ROC moves down/right (AUC $0.9959 \to 0.9942$) and the best achievable error rises by about 25% relative ($0.0259 \to 0.0325$; $0.0262 \to 0.0322$ with 2M samples).
- Here γ^* from priors and losses is no longer guaranteed optimal, because the NB ratio is not the true likelihood ratio, so thresholding it at γ^* is not the Bayes rule. Empirically the best threshold (0.64) is still close to γ^* and the error at γ^* is almost the same as the minimum.

Conclusion: yes, the mismatch hurts, but moderately. The decision boundary is still quadratic and uses the correct means and variances; it changes the shape and orientation of the quadratic discriminant in the (x_1, x_2, x_3) subspace. The correlations ($|\rho| \leq 0.5$) are moderate and the class means are well separated (distance 3), so the loss is noticeable but not dramatic.

Part C: Fisher LDA

Sample-average estimates from the 10K samples:

$$\hat{\mathbf{m}}_0 = (-0.500, -0.505, -0.496, -0.498)^T, \quad \hat{\mathbf{m}}_1 = (1.011, 1.016, 0.995, 0.983)^T,$$

and

$$\hat{C}_j = \tfrac{1}{N_j} \textstyle\sum_{n:L_n=j} (\mathbf{x}_n - \hat{\mathbf{m}}_j)(\mathbf{x}_n - \hat{\mathbf{m}}_j)^T$$

(both close to the true C_0, C_1). With equal weights, $S_B = (\hat{\mathbf{m}}_1 - \hat{\mathbf{m}}_0)(\hat{\mathbf{m}}_1 - \hat{\mathbf{m}}_0)^T$ and $S_W = \hat{C}_0 + \hat{C}_1$. The Fisher criterion $J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$ is maximized by the generalized eigenvector $S_B\mathbf{w} = \lambda S_W \mathbf{w}$ with the largest eigenvalue ($\lambda_{\max} = 4.689$; S_B has rank 1 so this is the only nonzero one). Normalized and oriented so that class 1 projects higher,

$$\mathbf{w}_{LDA} = (0.6028,\ 0.2747,\ 0.7086,\ 0.2432)^T,$$

which equals the closed form $S_W^{-1}(\hat{\mathbf{m}}_1 - \hat{\mathbf{m}}_0)/\|\cdot\|$ (checked numerically). The rule is $D = 1$ iff $\mathbf{w}_{LDA}^T \mathbf{x} > \tau$; sweeping τ over $(-\infty, \infty)$ (all projected values) gives the ROC in Fig. 3. The error at each τ is estimated by sample counts, $\#\{\text{errors}\}/N$.

Minimum $\hat{P}(\text{error}) = \mathbf{0.0521}$ at $\hat{\tau} \approx -0.0053$ ($P(D{=}1|L{=}1) = 0.951$, $P(D{=}1|L{=}0) = 0.058$), AUC $= 0.9854$.

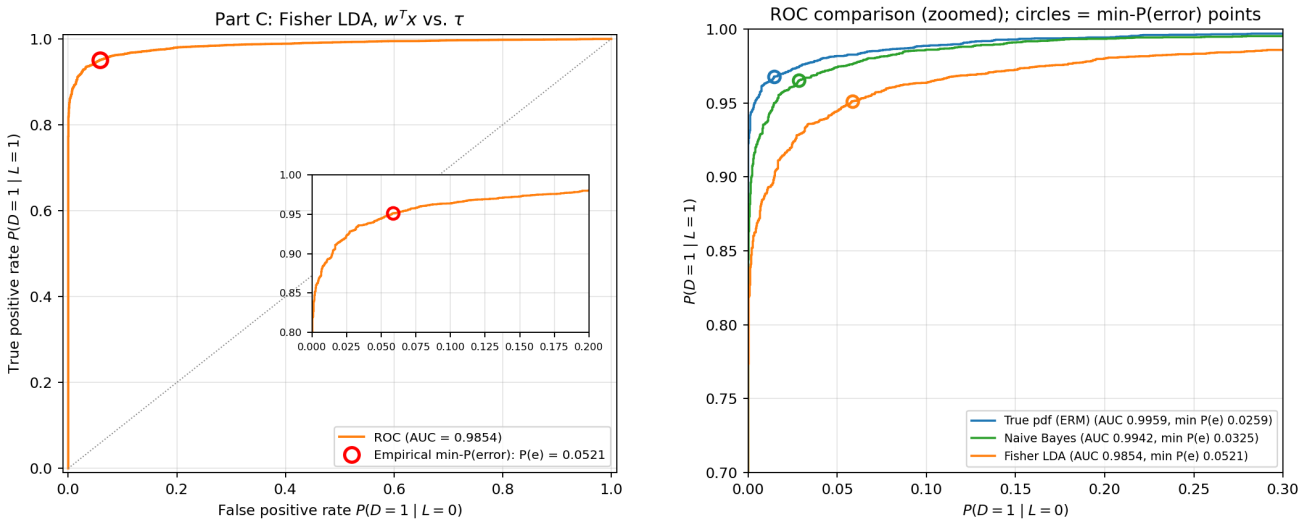


Figure 3: Left: Fisher LDA ROC with the minimum-$P(\text{error})$ point. Right: zoomed comparison of the three ROC curves.

Discussion. The ranking is true-pdf ERM (0.0259) $<$ naive Bayes (0.0325) $<$ LDA (0.0521), and the observed ROC curves show the same broad ordering (Fig. 3 right). LDA roughly doubles the error of the optimal classifier. The reason is structural: the Bayes rule is quadratic because C_1 has much larger variances than C_0 (e.g. $\text{var}(x_4)$ is 3 vs. 0.5), so the optimal boundary is a closed-ish surface wrapped around the tight class-0 cloud. LDA can only use a hyperplane, and Fisher's criterion maximizes projected mean separation relative to within-class spread; it does not generally minimize error. Under equal-covariance Gaussian models, the Fisher direction agrees with the Bayes linear discriminant when paired with the appropriate threshold. Naive Bayes, although its covariances are wrong, still exploits the covariance difference through a quadratic boundary, which is why it beats LDA here. As instructed, the LDA direction uses equal class weights; the unequal class frequencies still enter the sample-count error used to choose τ. LDA's advantages are that it needs no distributional knowledge and only d projection parameters.

Question 2: Decisions with unequal error costs

Data distribution

Four 3-D Gaussian components with means at the vertices of a regular tetrahedron of edge $a = 2.5$, so every pair of means is 2.5 apart:

$$\boldsymbol{\mu}_1 = (0,0,0),\ \boldsymbol{\mu}_2 = (2.5,0,0),\ \boldsymbol{\mu}_3 = (1.25,2.165,0),\ \boldsymbol{\mu}_4 = (1.25,0.722,2.041),$$

$$\Sigma_1 = \begin{bmatrix} 1 & .3 & -.2 \\ .3 & .8 & .1 \\ -.2 & .1 & 1.2 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} .9 & -.2 & .1 \\ -.2 & 1.1 & .3 \\ .1 & .3 & .8 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 1.2 & .2 & 0 \\ .2 & .9 & -.3 \\ 0 & -.3 & 1 \end{bmatrix}, \Sigma_4 = \begin{bmatrix} .8 & 0 & .2 \\ 0 & 1 & 0 \\ .2 & 0 & 1.1 \end{bmatrix}.$$

The average standard deviation is $\frac{1}{4}\sum_k \sqrt{\mathrm{tr}(\Sigma_k)/3} = 0.991$, so mean separations are $\approx 2.5\times$ the average standard deviation, giving substantial overlap, as requested. Class conditionals: $p(\mathbf{x}|L{=}1) = g(\mathbf{x}|\boldsymbol{\mu}_1, \Sigma_1)$, $p(\mathbf{x}|L{=}2) = g(\mathbf{x}|\boldsymbol{\mu}_2, \Sigma_2)$, $p(\mathbf{x}|L{=}3) = \frac{1}{2}g(\mathbf{x}|\boldsymbol{\mu}_3, \Sigma_3) + \frac{1}{2}g(\mathbf{x}|\boldsymbol{\mu}_4, \Sigma_4)$. Priors $(0.3, 0.3, 0.4)$.

Part A: MAP classification

1. 10,000 samples were generated (seed 2026) by drawing L, then the component (fair coin for class 3), then $\mathbf{x}$. Counts: $N_1 = 2950$, $N_2 = 2993$, $N_3 = 4057$.

2. Decision rule. With 0–1 loss, $R(D{=}i|\mathbf{x}) = 1 - P(L{=}i|\mathbf{x})$, so the minimum-$P$(error) (MAP) rule is

$$D(\mathbf{x}) = \arg\max_{j\in\{1,2,3\}} P(L{=}j|\mathbf{x}) = \arg\max_j \left[\ln p(\mathbf{x}|L{=}j) + \ln P(L{=}j)\right],$$

with $\ln p(\mathbf{x}|L{=}3)$ computed via log-sum-exp of the two components. The estimated confusion matrix, with entries $\hat{P}(D{=}i|L{=}j) = \#\{D{=}i, L{=}j\}/N_j$, is (counts in parentheses; columns sum to 1):

	$L=1$	$L=2$	$L=3$
$D=1$	**0.7963** (2349)	0.0672 (201)	0.0845 (343)
$D=2$	0.0600 (177)	**0.8199** (2454)	0.0656 (266)
$D=3$	0.1437 (424)	0.1129 (338)	**0.8499** (3448)

Estimated P(error) = **0.1749**. This is also the estimated minimum expected risk under 0–1 loss.

3. Visualization. Fig. 4: marker shape encodes the true label (dot $L{=}1$, triangle $L{=}2$, square $L{=}3$); green means correct, red incorrect. Errors concentrate in the central region where all four components overlap.

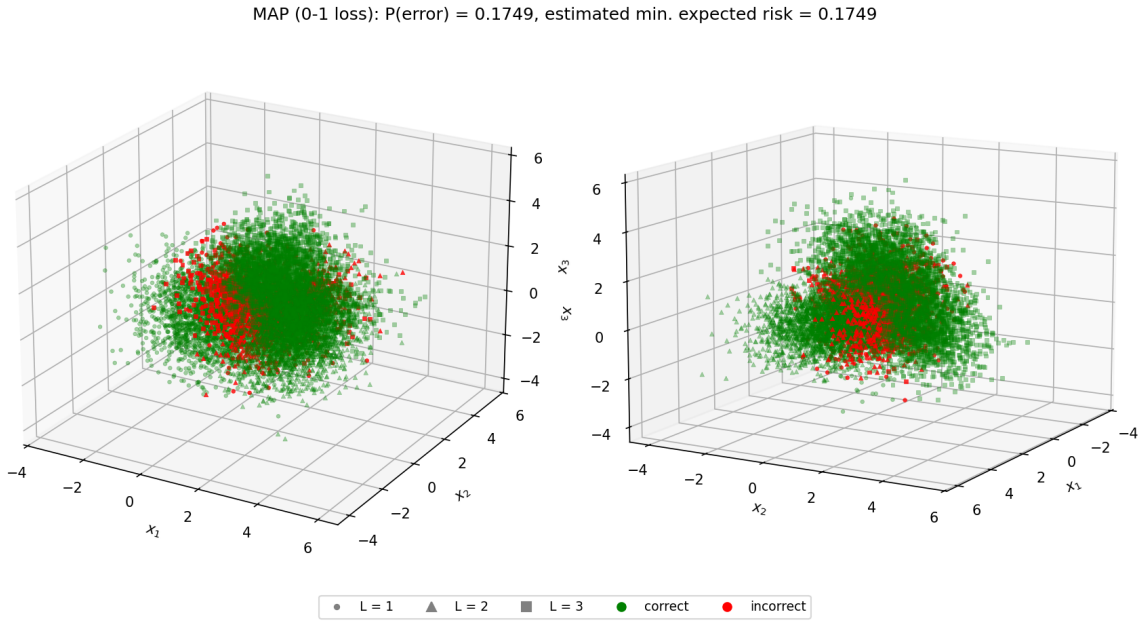


Figure 4: MAP classification of the 10K samples, two viewing angles.

Part B: ERM with Λ_{10} and Λ_{100}

The ERM rule is $D(\mathbf{x}) = \arg\min_i \sum_j \lambda_{ij} P(L{=}j|\mathbf{x})$. For $\Lambda_w = \begin{bmatrix} 0 & 1 & w \\ 1 & 0 & w \\ 1 & 1 & 0 \end{bmatrix}$ (with $P_j \equiv P(L{=}j|\mathbf{x})$):

$$R(1|\mathbf{x}) = P_2 + wP_3, \quad R(2|\mathbf{x}) = P_1 + wP_3, \quad R(3|\mathbf{x}) = P_1 + P_2 = 1 - P_3.$$

Between 1 and 2 the wP_3 term cancels, so that choice is still by larger posterior. Class 3 is chosen iff $1 - P_3 < \min(P_1, P_2) + wP_3$, i.e.

$$P(L{=}3|\mathbf{x}) > \frac{1 - \min\{P(L{=}1|\mathbf{x}), P(L{=}2|\mathbf{x})\}}{w+1}.$$

The right side lies between $\frac{1}{2(w+1)}$ and $\frac{1}{w+1}$. So with $w = 10$, class 3 is chosen once its posterior exceeds at most $1/11 \approx 0.09$; with $w = 100$, once it exceeds at most $1/101 \approx 0.01$.

The minimum expected risk is estimated by the sample average of the loss actually incurred, $\hat{R}_{\min} = \frac{1}{N}\sum_n \lambda_{D(\mathbf{x}_n),L_n}$. A second, lower-variance estimate is $\frac{1}{N}\sum_n \min_i R(D{=}i|\mathbf{x}_n)$, which uses the true posteriors. Both agree closely:

Loss	$\hat{R}_{\min} = \frac{1}{N}\sum_n \lambda_{D_n L_n}$	$\frac{1}{N}\sum_n \min_i R(D{=}i\|x_n)$	$\hat{P}$(error)	$\#\{D = 1, 2, 3\}$
0–1	0.1749	0.1775	0.1749	2893, 2897, 4210
Λ_{10}	0.3387	0.3451	0.2757	1567, 1933, 6500
Λ_{100}	0.4882	0.4899	0.4387	550, 1063, 8387

Confusion matrices $\hat{P}(D{=}i|L{=}j)$ for Λ_{10} and Λ_{100}, respectively:

	$L=1$	$L=2$	$L=3$
$D=1$	**0.4912** (1449)	0.0267 (80)	0.0094 (38)
$D=2$	0.0319 (94)	**0.6037** (1807)	0.0079 (32)
$D=3$	0.4769 (1407)	0.3695 (1106)	**0.9827** (3987)

	$L=1$	$L=2$	$L=3$
$D=1$	**0.1797** (530)	0.0060 (18)	0.0005 (2)
$D=2$	0.0098 (29)	**0.3445** (1031)	0.0007 (3)
$D=3$	0.8105 (2391)	0.6495 (1944)	**0.9988** (4052)

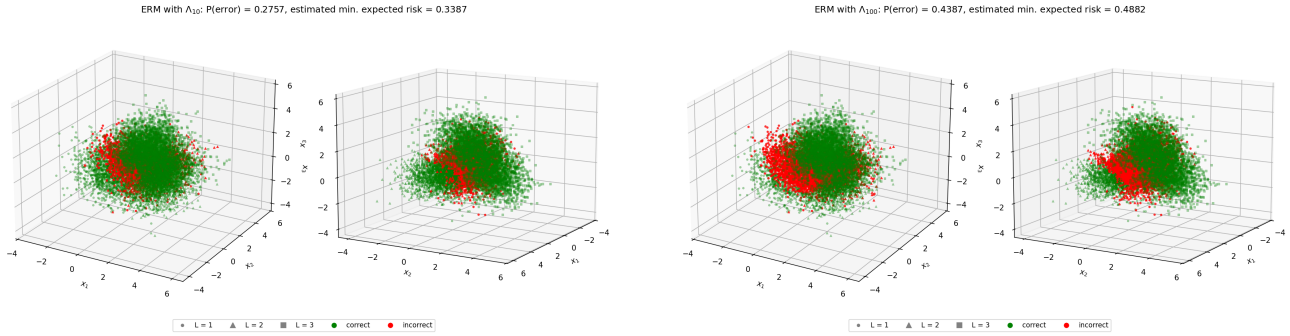


Figure 5: ERM decisions with Λ_{10} (left) and Λ_{100} (right); same marker/colour convention as Fig. 4.

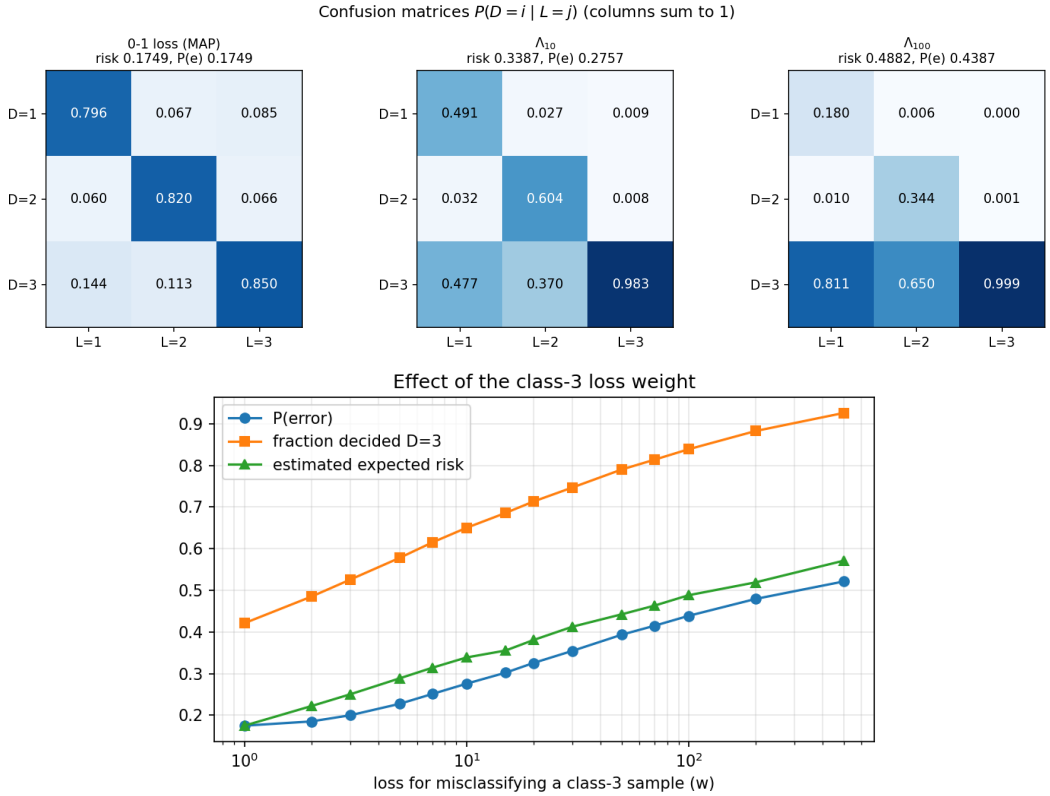


Figure 6: Top: confusion matrices for the three loss matrices. Bottom: P(error), fraction of $D = 3$ decisions, and estimated expected risk as the class-3 loss weight w varies from 1 (0–1 loss) to 500.

Insights.

- *Risk vs. error.* As w grows, ERM trades many cheap errors for few expensive ones. Class-3 recall goes $0.850 \to 0.983 \to 0.999$, while recall of classes 1 and 2 collapses ($0.796 \to 0.491 \to 0.180$ and $0.820 \to 0.604 \to 0.345$). The number of $D = 3$ decisions rises from 4210 to 6500 to 8387. $P(\text{error})$ rises sharply ($0.175 \to 0.276 \to 0.439$), but this is the risk-optimal behaviour, not a failure.
- *Minimum risk grows with w* ($0.175 \to 0.339 \to 0.488$): larger penalties cannot reduce the best achievable risk. However, the growth is strongly sub-linear in w because the classifier adapts. Under Λ_{100} only 5 of 4057 class-3 samples are missed, yet those 5 still contribute $500/10^4 = 0.05$ to the risk. The 4335 class-1/2 observations assigned class 3 contribute 0.4335, and 47 class-1/2 cross-errors contribute 0.0047, giving total risk 0.4882.
- *Asymmetry between classes 1 and 2:* class 1 loses more recall than class 2. Because of the covariance orientations, more of class 1's probability mass lies where $P(L{=}3|\mathbf{x})$ exceeds the low class-3 threshold.
- In the limit $w \to \infty$ the rule degenerates to "always decide 3", with risk 0.6 (all class-1/2 samples wrong at cost 1). The sweep shows the risk approaching that value (0.571 at $w = 500$).

Question 3: Estimated Gaussian class models

Model, estimators and decision rule

For each dataset with classes $j \in \mathcal{C}$ and samples $\{(\mathbf{x}_n, L_n)\}_{n=1}^N$, $\mathbf{x}_n \in \mathbb{R}^d$:

1. **Assumptions.** Samples are i.i.d.; the class-conditional pdf of each class is a single multivariate Gaussian, $p(\mathbf{x}|L{=}j) = g(\mathbf{x}|\mathbf{m}_j, C_j)$, with its own full covariance (heteroscedastic, i.e. a quadratic-discriminant model). No independence between features is assumed.
2. **Preprocessing.** For the required all-sample fit, each feature is z-scored with the mean and standard deviation of all samples, $\tilde{x}_k = (x_k - \bar{x}_k)/s_k$. An invertible affine map changes every $\ln p(\mathbf{x}|j)$ by the same constant (the log-Jacobian) and leaves an *unregularized* Gaussian classifier's decisions unchanged. Regularization with λI is *not* affine-invariant, though, and wine features differ in variance by $\sim 10^8$ (e.g. total SO_2 vs. density). Standardizing makes λI an isotropic shrinkage in comparable units. Without it, the same classifier on raw wine features has training error 0.600 instead of 0.503, because λ swamps the small-variance features. When tuning α (below), the mean and standard deviation are estimated on the training portion only and then applied to the validation portion, so no validation data leak into the fitted model.
3. **Estimates (maximum likelihood / sample averages).** $\hat{P}(L{=}j) = N_j/N$; $\hat{\mathbf{m}}_j = \frac{1}{N_j}\sum_{n:L_n=j} \mathbf{x}_n$; $\hat{C}_j = \frac{1}{N_j}\sum_{n:L_n=j}(\mathbf{x}_n - \hat{\mathbf{m}}_j)(\mathbf{x}_n - \hat{\mathbf{m}}_j)^T$.
4. **Regularization** (per class, following the hint): $C_j^{reg} = \hat{C}_j + \lambda_j I$ with $\lambda_j = \alpha\ \mathrm{tr}(\hat{C}_j)/\,\mathrm{rank}(\hat{C}_j)$, the arithmetic mean of the nonzero eigenvalues scaled by α. All eigenvalues of C_j^{reg} are then at least $\lambda_j > 0$.
5. **Decision rule** (0–1 loss, MAP): $D(\mathbf{x}) = \arg\max_j \big[-\frac{1}{2}(\mathbf{x} - \hat{\mathbf{m}}_j)^T (C_j^{reg})^{-1}(\mathbf{x} - \hat{\mathbf{m}}_j) - \frac{1}{2}\ln|C_j^{reg}| + \ln \hat{P}(L{=}j)\big]$, evaluated with Cholesky factors.
6. **Choosing α.** Training error reuses the fitting observations and can favor overfitting; it is not a reliable criterion for generalization. It need not decrease monotonically as α decreases. I therefore selected α on validation data and then, as the assignment requires, re-estimated all parameters on all samples and report the all-sample (training) error and confusion matrix. Wine: stratified 5-fold cross-validation, giving $\alpha = 10^{-3}$ (CV error is flat, about 0.515, for all $\alpha \le 10^{-2}$). HAR: the dataset's own subject-disjoint train/test split, used here as a validation split (train on the 7352 training samples, score the 2947 others), giving $\alpha = 10^{-4}$. Because this split chose α, its score is a validation score, not an untouched test score; no separate final test was performed. The sweep is in Table 2 and Fig. 7.

α	Wine: training	Wine: 5-fold CV	HAR: training	HAR: validation
1e-06	0.5002	0.5155	0.0217	0.0845
1e-05	0.5002	0.5155	0.0118	0.0821
0.0001	0.5008	0.5157	0.0030	0.0818
0.001	0.5033	0.5149	0.0149	0.1035
0.01	0.5043	0.5184	0.0652	0.1256
0.03	0.5065	0.5214	0.0938	0.1354
0.1	0.5159	0.5296	0.1482	0.1927
0.3	0.5208	0.5300	0.2633	0.3156

Table 2: P(error) versus regularization strength α.

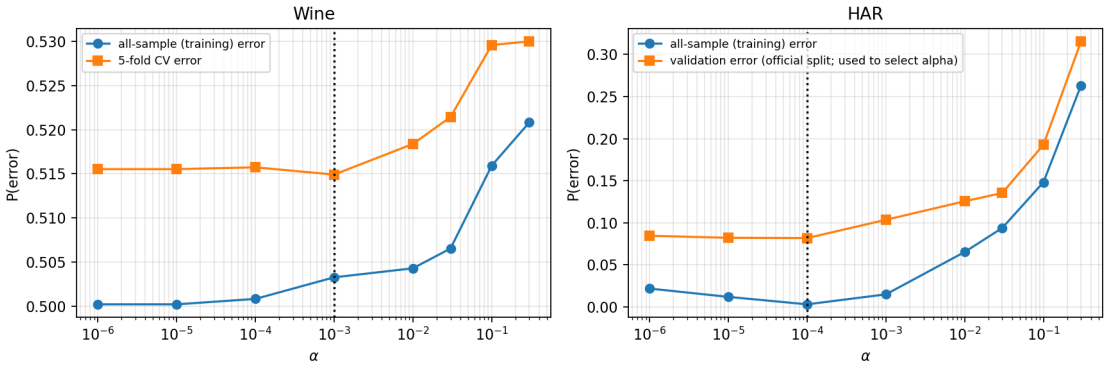


Figure 7: Effect of α; dotted line = selected value.

Results: Wine Quality (white, $N = 4898$, $d = 11$)

Seven quality levels occur (3–9); levels 0–2 and 10 have no samples and are therefore not classes of the model. Per-class estimates:

Class	N_j	$\hat{P}(L{=}j)$	rank($\hat{C}_j$)	λ_j	cond($\hat{C}_j$)	cond($\hat{C}_j + \lambda_j I$)
3	20	0.0041	11	3.47e-03	1.31e+03	1.07e+03
4	163	0.0333	11	1.36e-03	1.54e+02	1.47e+02
5	1457	0.2975	11	9.93e-04	2.18e+02	2.03e+02
6	2198	0.4488	11	9.37e-04	1.11e+02	1.07e+02
7	880	0.1797	11	7.71e-04	3.06e+02	2.83e+02
8	175	0.0357	11	8.86e-04	5.42e+02	4.74e+02
9	5	0.0010	4	1.28e-03	6.63e+17	2.54e+03

Class 9 has only 5 samples in 11 dimensions, so $\hat{C}_9$ has rank 4 and is singular (condition number $\sim 10^{17}$). Regularization is mandatory here; the other classes are well conditioned.

Estimated P(error) = 0.5033 ($\alpha = 10^{-3}$, all samples). Confusion matrix $\hat{P}(D{=}i|L{=}j)$:

	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$	$L=9$
$D=3$	**0.450**	0.018	0.006	0.002	0.000	0.000	0.000
$D=4$	0.050	**0.233**	0.036	0.012	0.002	0.000	0.000
$D=5$	0.150	0.417	**0.570**	0.221	0.032	0.017	0.000
$D=6$	0.200	0.252	0.279	**0.405**	0.216	0.280	0.000
$D=7$	0.100	0.074	0.107	0.353	**0.733**	0.617	0.000
$D=8$	0.050	0.006	0.001	0.007	0.017	**0.086**	0.000
$D=9$	0.000	0.000	0.000	0.000	0.000	0.000	**1.000**

For reference: always guessing the majority class (6) gives error 0.551; predicting within ± 1 grade succeeds for 92.7% of samples.

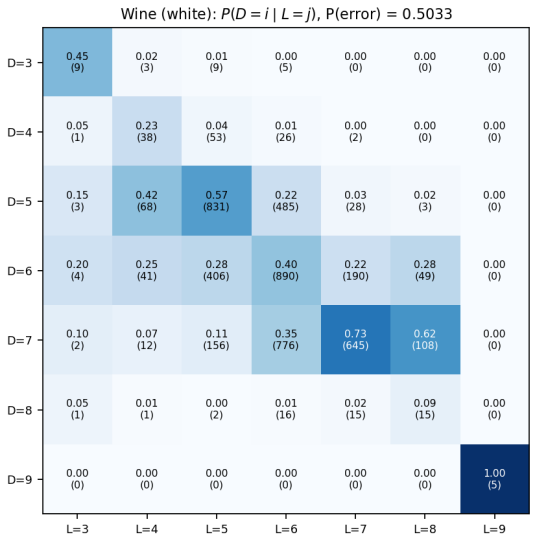


Figure 8: Wine confusion matrix (counts in parentheses).

Results: Human Activity Recognition ($N = 10299$, $d = 561$)

Class	N_j	$\hat{P}(L{=}j)$	rank($\hat{C}_j$)	λ_j	cond($\hat{C}_j$)	cond($\hat{C}_j + \lambda_j I$)
1 Walk	1722	0.1672	468	7.35e-05	7.0e+34	1.15e+06
2 Up	1544	0.1499	468	6.48e-05	3.1e+35	9.19e+05
3 Down	1406	0.1365	469	1.21e-04	1.6e+34	1.50e+06
4 Sit	1777	0.1725	468	3.80e-05	1.9e+34	1.15e+06
5 Stand	1906	0.1851	468	3.22e-05	7.6e+33	9.57e+05
6 Lay	1944	0.1888	468	4.96e-05	6.8e+34	1.05e+06

Every class covariance has numerical rank 468–469 although each class has more observations than features. The numerical rank shows strong linear redundancy among the 561 engineered features (many are derived from the same underlying signals); the specific relationships were not analysed. The raw $\hat{C}_j$ are therefore numerically singular (condition numbers $\sim 10^{35}$). Regularization brings them to $\sim 10^{6}$.

Estimated $P(\text{error}) = 0.0030$ ($\alpha = 10^{-4}$, all samples, 31 errors). Confusion matrix:

	L=Walk	L=Up	L=Down	L=Sit	L=Stand	L=Lay
D=Walk	**0.9994**	0.0000	0.0000	0.0000	0.0000	0.0000
D=Up	0.0006	**1.0000**	0.0085	0.0000	0.0000	0.0000
D=Down	0.0000	0.0000	**0.9915**	0.0000	0.0000	0.0000
D=Sit	0.0000	0.0000	0.0000	**0.9983**	0.0079	0.0000
D=Stand	0.0000	0.0000	0.0000	0.0017	**0.9921**	0.0000
D=Lay	0.0000	0.0000	0.0000	0.0000	0.0000	**1.0000**

On the official split's second part (train on 7352 samples, score the other 2947, from different subjects) the same model has validation error 0.082. This is optimistic as a test estimate, because the same split chose α.

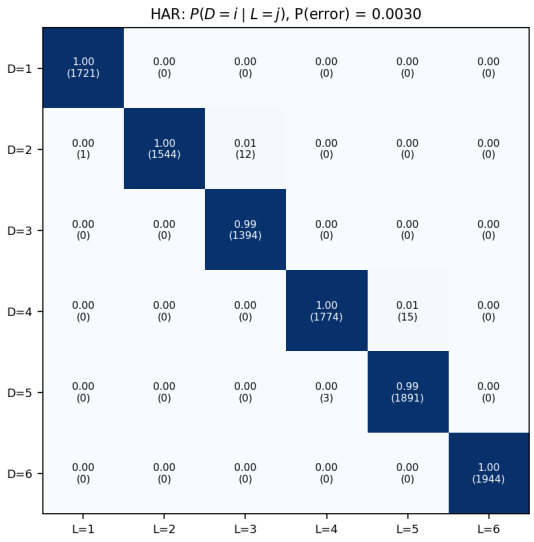


Figure 9: HAR confusion matrix (counts in parentheses).

Visualizations

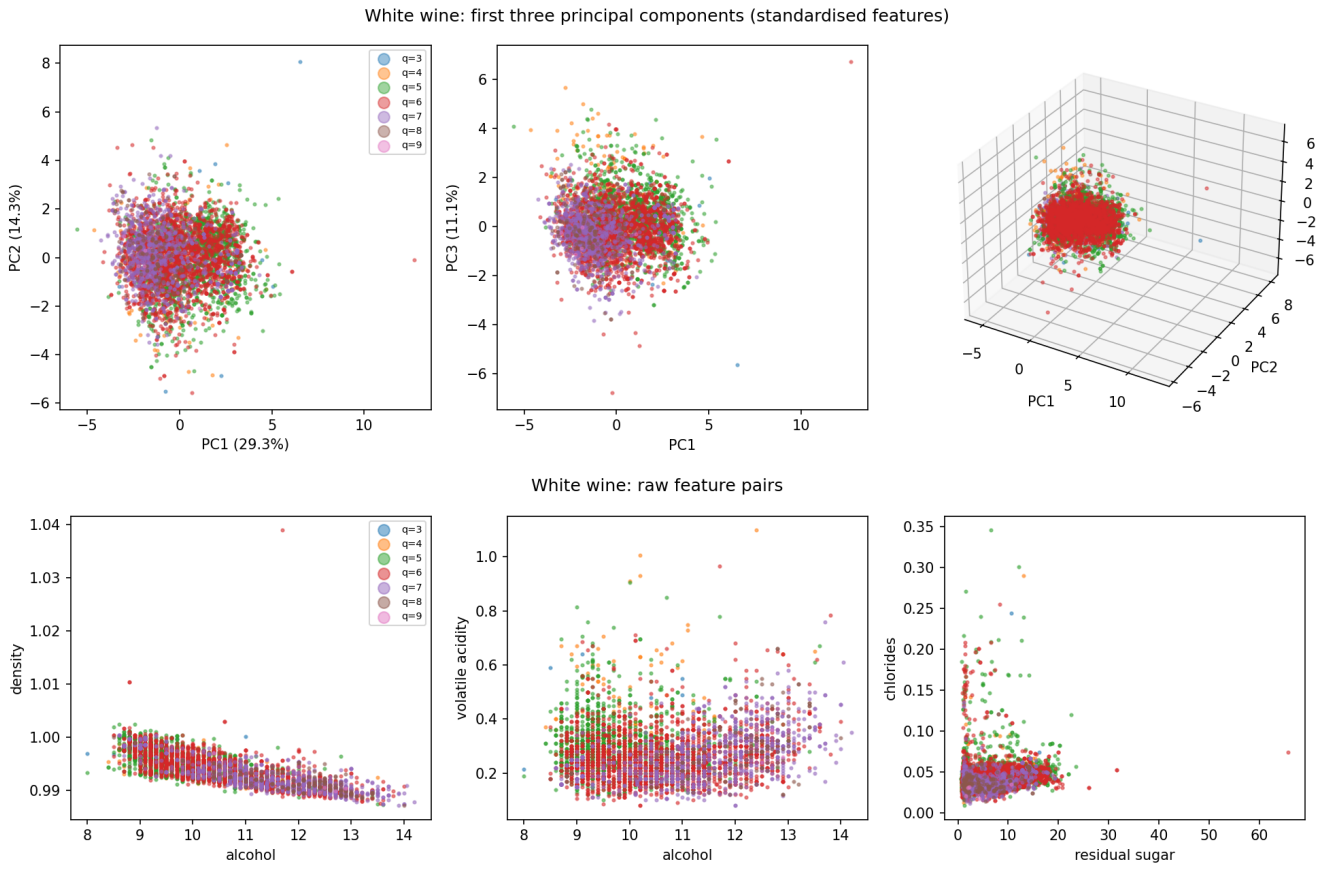


Figure 10: Wine: first three principal components of the standardized features (PC1–3 explain 29%, 14%, 11% of variance), and three raw feature pairs.

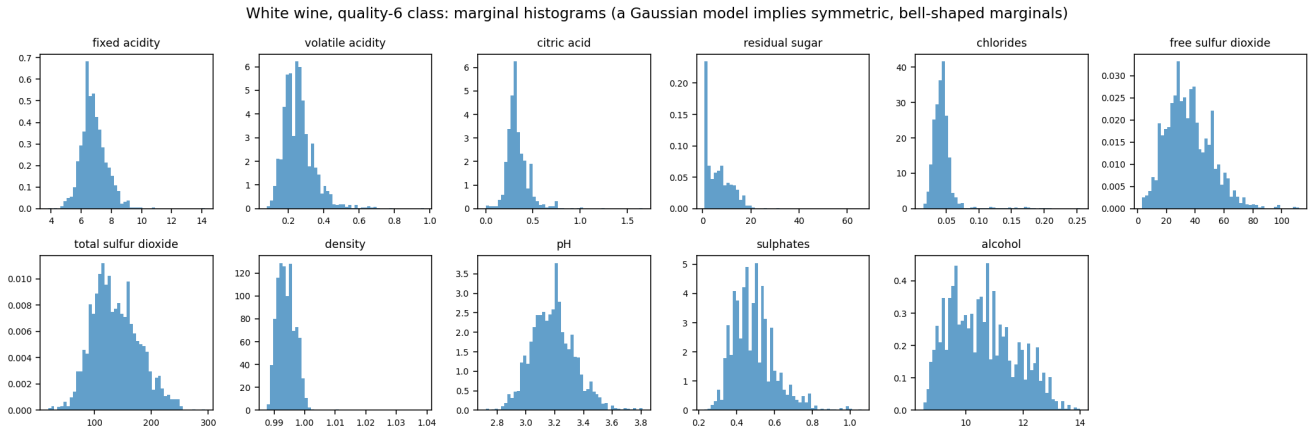


Figure 11: Wine: marginal histograms of all 11 features within the quality-6 class.

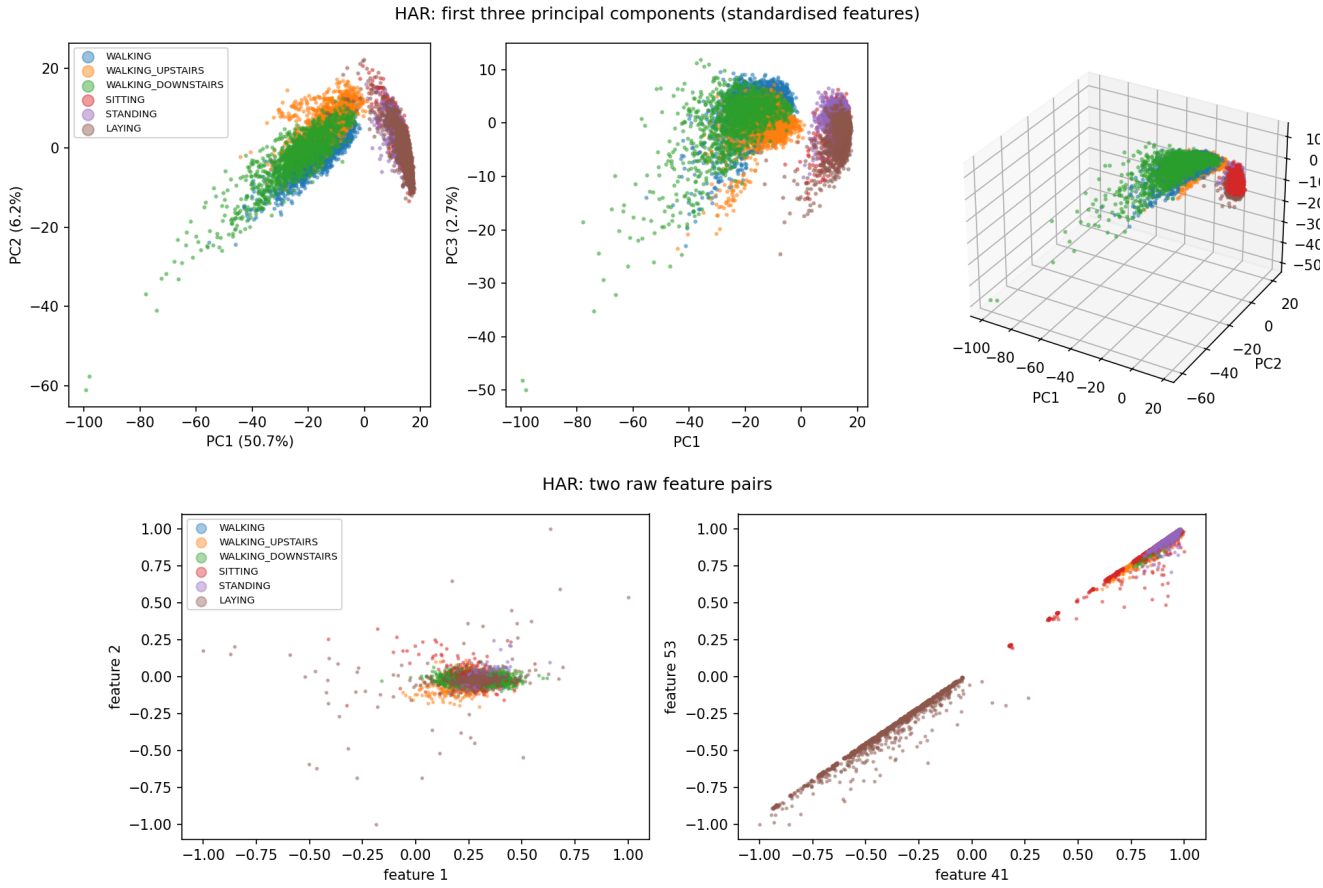


Figure 12: HAR: first three principal components (PC1 alone explains 50.7% of variance) and two raw feature pairs.

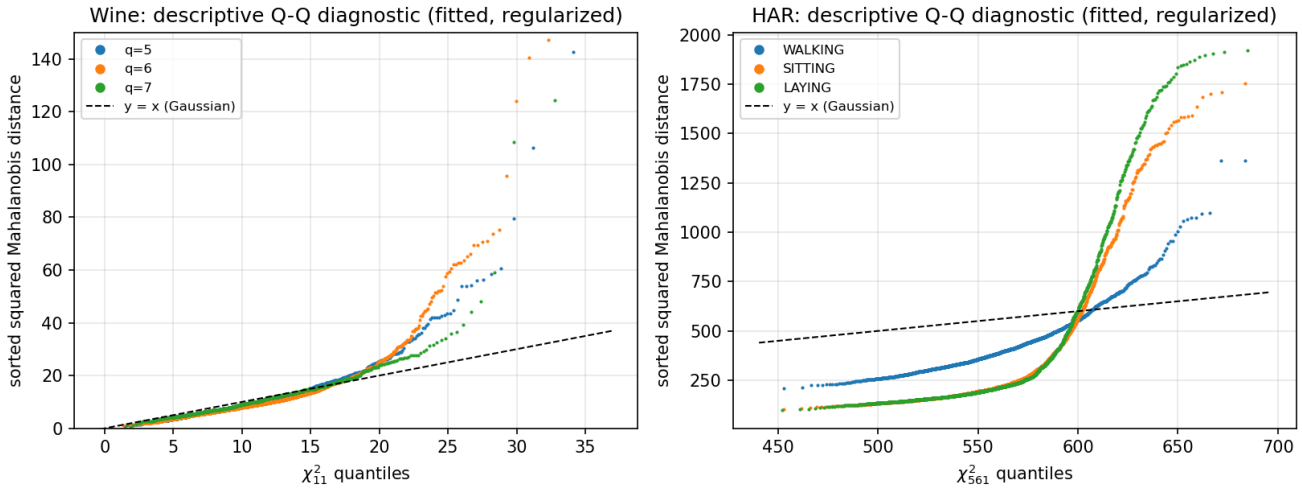


Figure 13: Gaussianity check (descriptive). If $\mathbf{x} \sim \mathcal{N}(\mathbf{m}_j, C_j)$, the squared Mahalanobis distance $r^2 = (\mathbf{x} - \mathbf{m}_j)^T C_j^{-1} (\mathbf{x} - \mathbf{m}_j) \sim \chi^2_d$. The plot shows sorted r^2 (with the fitted, regularized model) against χ^2_d quantiles; with known, full-rank Gaussian parameters the population reference is the dashed line. Fitting, regularization, and numerical rank deficiency alter that reference; this figure is descriptive, not a calibrated goodness-of-fit test.

Discussion: are Gaussian class conditionals appropriate?

Wine — poorly, and the problem itself is hard.

- *Non-Gaussian features.* Several marginals are strongly right-skewed or heavy-tailed (residual sugar, chlorides, free SO_2, volatile acidity; Fig. 11). The Q-Q plot (Fig. 13, left) follows χ^2_{11} up to roughly the 80th percentile, then shows much heavier tails: large distances suggest heavy tails, but fitted and regularized distances do not follow the nominal reference exactly. Transformations or mixture/robust models are plausible alternatives that would require separate evaluation.
- *Overlap.* In every projection (Fig. 10) the quality classes overlap almost completely and differ mostly by small mean shifts along alcohol/density. The labels are subjective, ordinal taster scores, which may limit predictability. These experiments do not estimate the true Bayes error; the fitted confusion matrix shows that nearly all errors are to adjacent grades (within ± 1: 92.7%), and the Gaussian model's $P(\text{error}) \approx 0.50$ is only modestly better than the majority-class baseline (0.55).
- *How the model shaped the confusion matrix.* With near-identical means, the quadratic rule is driven largely by covariance differences and priors. Class 6 is the largest class (prior 0.45) but has only 40.5% recall; 22% and 35% of its samples are assigned to the neighbouring grades 5 and 7, whose Gaussians overlap it almost completely. Rare grades are handled very differently. Class 9 (5 samples) gets a tiny, regularized Gaussian that recognizes its own 5 training points (5/5 correct), which is consistent with overfitting and provides little evidence about unseen quality-9 observations. Class 3 (20 samples) gets 45% recall on training data. With 77 parameters per class ($11 + 66$), these rare-class estimates have very high variance, and their training-set performance is not trustworthy. CV error (0.515) slightly above the training error (0.503) shows a modest measured gap, without establishing the extent of overfitting for every class.

HAR — not Gaussian, but the classes are so well separated that a Gaussian quadratic classifier still works very well.

- *Non-Gaussianity.* The features are bounded in $[-1, 1]$ by construction and come from many subjects, so each class may combine subject-specific distributions. The Q-Q plot (Fig. 13, right) is far from χ^2_{561}: distances are too small for most samples (the data effectively live in a ≈ 468-dimensional subspace, while regularization changes the distance metric) and far too large in the tail. The plot is descriptive, not a calibrated test: fitting, shrinkage and numerical rank deficiency can themselves produce departures from the nominal reference. The bounded features and heterogeneous activity structure also make an exact Gaussian model implausible; the plot alone cannot separate these effects.
- *Separation.* PCA (Fig. 12) shows two groups split cleanly along PC1: dynamic activities (walking, upstairs, downstairs) and static ones (sitting, standing, laying). Laying is also distinct. The only confusions are within

the static group (sitting↔standing: 18 samples) and within the dynamic group (12 downstairs and 1 walking sample decided as upstairs), exactly the physically similar pairs. In 561 dimensions, a quadratic boundary with class-specific covariances is flexible enough to separate these classes even though each class is not Gaussian.

- *Overfitting and the role of λ.* Each class model has $561 + 561 \cdot 562/2 \approx 158{,}000$ parameters estimated from $\sim 1{,}700$ samples. The all-sample error of 0.30% is therefore heavily optimistic: on the other subjects of the official split the validation error is 8.2%. Table 2 shows the bias–variance trade-off. Large α (strong shrinkage toward a spherical covariance) underfits (error grows to 26–32% at $\alpha = 0.3$). Tiny α leaves the smallest eigendirections, which are poorly estimated, almost unregularized. Training error need not be monotone in α (here it is lowest at $\alpha = 10^{-4}$), and validation error is best at intermediate $\alpha = 10^{-4}$ to 10^{-5}.

Summary. The minimum-P(error) rule is guaranteed optimal under the specified class densities and priors; model mismatch can remove that guarantee. For wine, model mismatch (skewed, heavy-tailed, almost fully overlapping classes) plus severe class imbalance yields $\approx$ 50% error concentrated between adjacent grades. For HAR, the Gaussian model is also mismatched, but the class-conditional densities are concentrated in different regions, so the induced quadratic boundaries still achieve near-zero training error and $\approx$ 8% validation error. In both cases regularization was essential: it made singular covariance estimates invertible (wine class 9; all HAR classes), and its strength set the bias–variance balance.

Data provenance and verification

The instructor's *EECE5644 Fall 2026 Assignment 1* supplies the tasks and Question 1 parameters. The original datasets were downloaded from:

- UCI Wine Quality: `https://archive.ics.uci.edu/static/public/186/wine+quality.zip`.
- UCI Human Activity Recognition: `https://archive.ics.uci.edu/static/public/240/human+activity+recognition+using+smartphones.zip`.

Archive SHA-256 values (also stored in `data/sources.json`):

```
wine:
3ed56667f4b828242bd732d7d1dd7f28
61e54432239d7fa63877014cbb0304d4
har:
c00b803081a5c797cd5e4b83700a9810
b38d53d9d84e01917e090e1fdbc81031
```

All three experiments were rerun. Twenty-two checks passed, covering Gaussian log-density agreement with SciPy, tied-score ROC counts, an independent rank-based AUC calculation, the LDA closed form, and consistency of errors and risks with confusion counts. These checks support numerical consistency; they do not prove the modeling assumptions. The prescribed all-sample errors are training errors; the additional validation calculations are identified separately.

Appendix: Code

The following Python source implements the three experiments. From the project folder, run `python run_all.py`; it downloads data when absent, executes all questions, and runs the numerical checks. `python gen_tables.py` regenerates every table in this report from the saved JSON summaries. `python build_latex.py` compiles this report with Tectonic or pdfLaTeX. Supporting scripts and the independent study guide are supplied with the repository.

common.py

```python
"""Shared helpers for EECE5644 Assignment 1."""
import numpy as np
from scipy.linalg import solve_triangular


def gauss_logpdf(X, m, C):
    """Log of the multivariate Gaussian pdf g(x|m,C), evaluated row-wise on X (N x d).
    Uses a Cholesky factorisation for numerical stability."""
    X = np.atleast_2d(X)
    d = X.shape[1]
    L = np.linalg.cholesky(C)
    Z = solve_triangular(L, (X - m).T, lower=True, check_finite=False) # d x N, Z = L^{-1}(x - m)
    maha = np.sum(Z ** 2, axis=0) # (x-m)^T C^{-1} (x-m)
    logdet = 2.0 * np.sum(np.log(np.diag(L)))
    return -0.5 * (d * np.log(2 * np.pi) + logdet + maha)


def roc_sweep(score, labels):
    """Sweep a threshold t over all observed score values for the rule D = 1 iff score > t.
    Returns thresholds (starting at -inf, so the curve starts at (1,1)), P(D=1|L=0) and P(D=1|L=1).
    The largest observed score gives the finite end point (0,0)."""
    score = np.asarray(score, float)
    labels = np.asarray(labels).astype(int)
    n1, n0 = labels.sum(), (1 - labels).sum()
    order = np.argsort(score)
    s_sorted = score[order]
    l_sorted = labels[order]
    # number of samples with score <= s_sorted[k] -> those are decided 0 at threshold s_sorted[k]
    c1_le = np.cumsum(l_sorted)
    c0_le = np.cumsum(1 - l_sorted)
    # keep only the last index of each run of tied scores
    last = np.r_[s_sorted[1:] != s_sorted[:-1], True]
    thr = np.r_[-np.inf, s_sorted[last]]
    tpr = np.r_[1.0, (n1 - c1_le[last]) / n1]
    fpr = np.r_[1.0, (n0 - c0_le[last]) / n0]
    return thr, fpr, tpr


def auc(fpr, tpr):
    # Preserve the order of vertical ROC segments (tied false-positive rates).
    return -np.trapezoid(tpr, fpr)
```

q1.py

```python
"""Question 1: ERM / LRT with true pdf, Naive Bayes, and Fisher LDA on a 2-class 4-D Gaussian problem."""
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.linalg import eigh
from common import gauss_logpdf, roc_sweep, auc

rng = np.random.default_rng(5644)
N = 10000
P = np.array([0.35, 0.65])
m0 = -0.5 * np.ones(4)
C0 = 0.25 * np.array([[2, -0.5, 0.3, 0],
                      [-0.5, 1, -0.5, 0],
                      [0.3, -0.5, 1, 0],
                      [0, 0, 0, 2]])
m1 = np.ones(4)
C1 = np.array([[1, 0.3, -0.2, 0],
               [0.3, 2, 0.3, 0],
               [-0.2, 0.3, 1, 0],
```

```
            [0, 0, 0, 3]])
for C in (C0, C1):
    assert np.all(np.linalg.eigvalsh(C) > 0)


def sample(n, rng):
    L = (rng.random(n) >= P[0]).astype(int) # P(L=1)=0.65
    X = np.empty((n, 4))
    X[L == 0] = rng.multivariate_normal(m0, C0, (L == 0).sum())
    X[L == 1] = rng.multivariate_normal(m1, C1, (L == 1).sum())
    return X, L


X, L = sample(N, rng)
np.savez("out/q1_data.npz", X=X, L=L) # same data set used for every part
N0, N1 = (L == 0).sum(), (L == 1).sum()
res = {"N0": int(N0), "N1": int(N1)}

gamma_theory = P[0] / P[1] # 0-1 loss: (l10-l00)P0 / ((l01-l11)P1)
res["gamma_theory"] = gamma_theory


def analyse(score, name, theory_thr=None, priors=P):
    """score: decision statistic, D=1 iff score > thr. Returns dict + ROC arrays."""
    thr, fpr, tpr = roc_sweep(score, L)
    perr = fpr * priors[0] + (1 - tpr) * priors[1]
    k = int(np.argmin(perr))
    # any threshold in [thr_k, thr_{k+1}) gives the same decisions; report the midpoint
    t_lo = thr[k]
    t_hi = thr[k + 1] if k + 1 < len(thr) else np.inf
    t_mid = 0.5 * (t_lo + t_hi) if np.isfinite(t_lo) and np.isfinite(t_hi) else t_lo
    out = dict(name=name, k=k, thr_mid=float(t_mid), thr_lo=float(t_lo), thr_hi=float(t_hi),
               perr_min=float(perr[k]), tpr=float(tpr[k]), fpr=float(fpr[k]),
               auc=float(auc(fpr, tpr)), n_thr=len(thr))
    if theory_thr is not None:
        D = score > theory_thr
        out["theory_tpr"] = float(D[L == 1].mean())
        out["theory_fpr"] = float(D[L == 0].mean())
        out["theory_perr"] = float(out["theory_fpr"] * P[0] + (1 - out["theory_tpr"]) * P[1])
    return out, (fpr, tpr, perr, thr)


# ---------- Part A: true pdf ----------
llr_A = gauss_logpdf(X, m1, C1) - gauss_logpdf(X, m0, C0) # ln p(x|1) - ln p(x|0)
rA, curA = analyse(llr_A, "True pdf (ERM)", np.log(gamma_theory))
rA["gamma_emp"] = float(np.exp(rA["thr_mid"]))
rA["gamma_emp_lo"] = float(np.exp(rA["thr_lo"]))
rA["gamma_emp_hi"] = float(np.exp(rA["thr_hi"]))

# ---------- Part B: naive Bayes (diagonal covariances with true variances) ----------
C0d, C1d = np.diag(np.diag(C0)), np.diag(np.diag(C1))
llr_B = gauss_logpdf(X, m1, C1d) - gauss_logpdf(X, m0, C0d)
rB, curB = analyse(llr_B, "Naive Bayes", np.log(gamma_theory))
rB["gamma_emp"] = float(np.exp(rB["thr_mid"]))
rB["gamma_emp_lo"] = float(np.exp(rB["thr_lo"]))
rB["gamma_emp_hi"] = float(np.exp(rB["thr_hi"]))

# ---------- Part C: Fisher LDA from sample estimates ----------
mu0_hat, mu1_hat = X[L == 0].mean(0), X[L == 1].mean(0)
S0_hat = np.cov(X[L == 0].T, bias=True) # sample-average (1/N_j) estimators
S1_hat = np.cov(X[L == 1].T, bias=True)
Sb = np.outer(mu1_hat - mu0_hat, mu1_hat - mu0_hat)
Sw = S0_hat + S1_hat # equal weights, priors ignored (as in class)
evals, evecs = eigh(Sb, Sw) # generalised eigenproblem Sb w = lambda Sw w
w = evecs[:, np.argmax(evals)]
w = w / np.linalg.norm(w)
if w @ mu1_hat < w @ mu0_hat: # orient so that class 1 projects higher
    w = -w
y = X @ w
rC, curC = analyse(y, "Fisher LDA", priors=np.array([N0, N1]) / N) # sample-count estimate
rC["count_check"] = float(np.mean((y > rC["thr_mid"]) != L))
w_closed = np.linalg.solve(Sw, mu1_hat - mu0_hat)
w_closed /= np.linalg.norm(w_closed)
res["lda"] = dict(w=w.tolist(), w_closed=w_closed.tolist(), eig=float(evals.max()),
                  mu0_hat=mu0_hat.tolist(), mu1_hat=mu1_hat.tolist(),
                  S0_hat=S0_hat.tolist(), S1_hat=S1_hat.tolist())

# ---------- reference: theoretical min P(error) via a large independent Monte Carlo ----------
Xbig, Lbig = sample(2_000_000, np.random.default_rng(1))
llr_big = gauss_logpdf(Xbig, m1, C1) - gauss_logpdf(Xbig, m0, C0)
res["perr_bayes_mc"] = float(np.mean((llr_big > np.log(gamma_theory)) != Lbig))
llr_bigB = gauss_logpdf(Xbig, m1, C1d) - gauss_logpdf(Xbig, m0, C0d)
res["perr_nb_mc_theorythr"] = float(np.mean((llr_bigB > np.log(gamma_theory)) != Lbig))

```

```
res["A"], res["B"], res["C"] = rA, rB, rC
for key, curve in [("A", curA), ("B", curB), ("C", curC)]:
    fpr, tpr, perr, thresholds = curve
    np.savez(f"out/q1_roc_{key}.npz", fpr=fpr, tpr=tpr, fnr=1-tpr, perr=perr, thresholds=thresholds)
res["perr_bayes_mc_se"] = float(np.sqrt(res["perr_bayes_mc"]*(1-res["perr_bayes_mc"])/len(Lbig)))

json.dump(res, open("out/q1_results.json", "w"), indent=2)

# ---------- figures ----------
def roc_plot(ax, cur, r, title, color, theory=True):
    fpr, tpr, _, _ = cur
    ax.plot(fpr, tpr, color=color, lw=1.6, label=f"ROC (AUC = {r['auc']:.4f})")
    ax.plot(r["fpr"], r["tpr"], "o", ms=10, mfc="none", mec="red", mew=2.2,
           label=f"Empirical min-P(error): P(e) = {r['perr_min']:.4f}")
    if theory and "theory_tpr" in r:
        ax.plot(r["theory_fpr"], r["theory_tpr"], "x", ms=10, color="black", mew=2.2,
                label=f"Theoretical $\\gamma^*=0.538$: P(e) = {r['theory_perr']:.4f}")
    ax.plot([0, 1], [0, 1], ":", color="gray", lw=0.8)
    ax.set_xlabel(r"False positive rate $P(D=1\mid L=0)$")
    ax.set_ylabel(r"True positive rate $P(D=1\mid L=1)$")
    ax.set_title(title)
    ax.legend(loc="lower right", fontsize=8.5)
    ax.grid(alpha=0.3)
    ax.set_xlim(-0.01, 1.01); ax.set_ylim(-0.01, 1.01)


for cur, r, t, c, fn in [(curA, rA, "Part A: ERM with true pdf", "C0", "q1a_roc"),
                         (curB, rB, "Part B: Naive Bayes (diagonal covariances)", "C2", "q1b_roc"),
                         (curC, rC, r"Part C: Fisher LDA, $w^T x$ vs. $\tau$", "C1", "q1c_roc")]:
    fig, ax = plt.subplots(figsize=(6.2, 5.2))
    roc_plot(ax, cur, r, t, c)
    # zoomed inset near the operating point
    ins = ax.inset_axes([0.45, 0.32, 0.5, 0.36])
    ins.plot(cur[0], cur[1], color=c, lw=1.4)
    ins.plot(r["fpr"], r["tpr"], "o", ms=8, mfc="none", mec="red", mew=2)
    if "theory_tpr" in r:
        ins.plot(r["theory_fpr"], r["theory_tpr"], "x", ms=8, color="black", mew=2)
    ins.set_xlim(0, 0.2); ins.set_ylim(0.8, 1.0); ins.grid(alpha=0.3); ins.tick_params(labelsize=7)
    fig.tight_layout(); fig.savefig(f"figs/{fn}.png", dpi=170); plt.close(fig)

# comparison plot
fig, ax = plt.subplots(figsize=(6.2, 5.2))
for cur, r, c in [(curA, rA, "C0"), (curB, rB, "C2"), (curC, rC, "C1")]:
    ax.plot(cur[0], cur[1], color=c, lw=1.5, label=f"{r['name']} (AUC {r['auc']:.4f}, min P(e) {r['perr_min']:.4f})")
    ax.plot(r["fpr"], r["tpr"], "o", ms=8, mfc="none", mec=c, mew=2)
ax.set_xlim(0, 0.3); ax.set_ylim(0.7, 1.0); ax.grid(alpha=0.3)
ax.set_xlabel(r"$P(D=1\mid L=0)$"); ax.set_ylabel(r"$P(D=1\mid L=1)$")
ax.set_title("ROC comparison (zoomed); circles = min-P(error) points")
ax.legend(fontsize=8, loc="lower right")
fig.tight_layout(); fig.savefig("figs/q1_compare.png", dpi=170); plt.close(fig)

# P(error) vs threshold
fig, axs = plt.subplots(1, 3, figsize=(13, 3.6))
for ax, cur, r, xl in [(axs[0], curA, rA, r"$\ln\gamma$"), (axs[1], curB, rB, r"$\ln\gamma$"),
                       (axs[2], curC, rC, r"$\tau$")]:
    _, _, perr, thr = cur
    ok = np.isfinite(thr)
    ax.plot(thr[ok], perr[ok], lw=1.2)
    ax.axvline(r["thr_mid"], color="red", ls="--", lw=1, label=f"empirical min at {r['thr_mid']:.3f}")
    if r is not rC:
        ax.axvline(np.log(gamma_theory), color="k", ls=":", lw=1.2, label=fr"$\ln\gamma^*$ = {np.log(gamma_theory):.3f}")
    lo, hi = np.percentile(thr[ok], [2, 98])
    ax.set_xlim(lo, hi); ax.set_xlabel(xl); ax.set_ylabel("P(error)"); ax.set_title(r["name"])
    ax.legend(fontsize=8); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig("figs/q1_perr_vs_thr.png", dpi=160); plt.close(fig)

print(json.dumps({k: v for k, v in res.items() if k != "lda"}, indent=1))
print("w", w, "w_closed", w_closed)
```

q2.py

```
"""Question 2: 3-class, 3-D Gaussian-mixture problem; MAP (0-1 loss) and ERM with Lambda_10, Lambda_100."""
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from common import gauss_logpdf

rng = np.random.default_rng(2026)
```

```
N = 10000
priors = np.array([0.3, 0.3, 0.4])

# Four component means on a regular tetrahedron with edge length a = 2.5
a = 2.5
means = np.array([[0, 0, 0],
                  [a, 0, 0],
                  [a / 2, a * np.sqrt(3) / 2, 0],
                  [a / 2, a * np.sqrt(3) / 6, a * np.sqrt(2 / 3)]])
covs = np.array([[[1.0, 0.3, -0.2], [0.3, 0.8, 0.1], [-0.2, 0.1, 1.2]],
                 [[0.9, -0.2, 0.1], [-0.2, 1.1, 0.3], [0.1, 0.3, 0.8]],
                 [[1.2, 0.2, 0.0], [0.2, 0.9, -0.3], [0.0, -0.3, 1.0]],
                 [[0.8, 0.0, 0.2], [0.0, 1.0, 0.0], [0.2, 0.0, 1.1]]])
for C in covs:
    assert np.all(np.linalg.eigvalsh(C) > 0)
# class 1 <- component 0, class 2 <- component 1, class 3 <- 0.5*comp 2 + 0.5*comp 3
avg_std = np.mean([np.sqrt(np.trace(C) / 3) for C in covs])
dists = [np.linalg.norm(means[i] - means[j]) for i in range(4) for j in range(i + 1, 4)]

# ---- 1. generate data ----
L = rng.choice([1, 2, 3], size=N, p=priors)
comp = np.where(L == 1, 0, np.where(L == 2, 1, 2 + (rng.random(N) < 0.5)))
X = np.empty((N, 3))
for c in range(4):
    idx = comp == c
    X[idx] = rng.multivariate_normal(means[c], covs[c], idx.sum())
np.savez("out/q2_data.npz", X=X, L=L, comp=comp)


def class_loglik(X):
    """N x 3 matrix of ln p(x|L=j)."""
    l1 = gauss_logpdf(X, means[0], covs[0])
    l2 = gauss_logpdf(X, means[1], covs[1])
    l3 = np.logaddexp(np.log(0.5) + gauss_logpdf(X, means[2], covs[2]),
                      np.log(0.5) + gauss_logpdf(X, means[3], covs[3]))
    return np.c_[l1, l2, l3]


ll = class_loglik(X)
lj = ll + np.log(priors) # ln p(x|j)P(j)
post = np.exp(lj - np.logaddexp.reduce(lj, axis=1, keepdims=True)) # P(L=j|x)

Lambda = {"0-1": 1 - np.eye(3),
          "10": np.array([[0, 1, 10], [1, 0, 10], [1, 1, 0]], float),
          "100": np.array([[0, 1, 100], [1, 0, 100], [1, 1, 0]], float)}

res = {"means": means.tolist(), "covs": covs.tolist(), "avg_std": avg_std,
       "pairwise_dist": dists, "class_counts": [int((L == j).sum()) for j in (1, 2, 3)]}

for key, Lam in Lambda.items():
    R = post @ Lam.T # R[n,i] = sum_j Lam[i,j] P(j|x_n) (conditional risk)
    D = np.argmin(R, axis=1) + 1
    conf = np.zeros((3, 3))
    counts = np.zeros((3, 3), int)
    for i in (1, 2, 3):
        for j in (1, 2, 3):
            counts[i - 1, j - 1] = np.sum((D == i) & (L == j))
            conf[i - 1, j - 1] = counts[i - 1, j - 1] / np.sum(L == j)
    risk = float(np.mean(Lam[D - 1, L - 1])) # sample average of incurred loss
    perr = float(np.mean(D != L))
    res[key] = dict(conf=conf.tolist(), counts=counts.tolist(), risk=risk, perr=perr,
                    dec_counts=[int((D == i).sum()) for i in (1, 2, 3)],
                    exp_risk_cond=float(np.mean(R.min(axis=1))))
    np.save(f"out/q2_D_{key}.npy", D)

    # ---- 3-D scatter plot ----
    fig = plt.figure(figsize=(12, 6.2))
    markers = {1: ".", 2: "^", 3: "s"}
    sizes = {1: 30, 2: 10, 3: 8}
    ok = D == L
    for p, (elev, azim) in enumerate([(22, -60), (12, 30)]):
        ax = fig.add_subplot(1, 2, p + 1, projection="3d")
        for j in (1, 2, 3):
            for good, col in ((True, "green"), (False, "red")):
                idx = (L == j) & (ok == good)
                ax.scatter(*X[idx].T, marker=markers[j], s=sizes[j],
                           c=col, alpha=0.35 if good else 0.8, linewidths=0.3)
        ax.view_init(elev, azim)
        ax.set_box_aspect(None, zoom=1.0)
        ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$"); ax.set_zlabel("$x_3$")
    handles = [Line2D([], [], marker=markers[j], ls="", color="gray", ms=7, label=f"L = {j}") for j in (1, 2, 3)]
    handles += [Line2D([], [], marker="o", ls="", color="green", label="correct"),
                Line2D([], [], marker="o", ls="", color="red", label="incorrect")]
    name = "MAP (0-1 loss)" if key == "0-1" else fr"ERM with $\Lambda_{{{key}}}$"
```

```
    fig.suptitle(f"{name}: P(error) = {perr:.4f}, estimated min. expected risk = {risk:.4f}")
    fig.legend(handles=handles, loc="lower center", ncol=5, fontsize=9)
    fig.subplots_adjust(left=0.02, right=0.98, top=0.95, bottom=0.07, wspace=0.08)
    fig.savefig(f"figs/q2_scatter_{key.replace('-', '')}.png", dpi=150); plt.close(fig)

# ---- confusion-matrix heat maps side by side ----
fig, axs = plt.subplots(1, 3, figsize=(12, 3.8))
for ax, key in zip(axs, Lambda):
    M = np.array(res[key]["conf"])
    ax.imshow(M, cmap="Blues", vmin=0, vmax=1)
    for i in range(3):
        for j in range(3):
            ax.text(j, i, f"{M[i, j]:.3f}", ha="center", va="center",
                    color="white" if M[i, j] > 0.6 else "black", fontsize=10)
    ax.set_xticks(range(3), ["L=1", "L=2", "L=3"]); ax.set_yticks(range(3), ["D=1", "D=2", "D=3"])
    t = "0-1 loss (MAP)" if key == "0-1" else fr"$\Lambda_{{{key}}}$"
    ax.set_title(f"{t}\nrisk {res[key]['risk']:.4f}, P(e) {res[key]['perr']:.4f}", fontsize=10)
fig.suptitle(r"Confusion matrices $P(D=i\mid L=j)$ (columns sum to 1)")
fig.tight_layout(); fig.savefig("figs/q2_confusions.png", dpi=150); plt.close(fig)

# ---- risk/error trade-off as the class-3 loss weight grows ----
ws = np.r_[1, 2, 3, 5, 7, 10, 15, 20, 30, 50, 70, 100, 200, 500]
rk, pe, p3 = [], [], []
for wgt in ws:
    Lam = np.array([[0, 1, wgt], [1, 0, wgt], [1, 1, 0]], float)
    D = np.argmin(post @ Lam.T, axis=1) + 1
    rk.append(np.mean(Lam[D - 1, L - 1])); pe.append(np.mean(D != L)); p3.append(np.mean(D == 3))
res["sweep"] = dict(w=ws.tolist(), risk=rk, perr=pe, frac_D3=p3)
fig, ax = plt.subplots(figsize=(6.4, 4))
ax.semilogx(ws, pe, "o-", label="P(error)")
ax.semilogx(ws, p3, "s-", label="fraction decided D=3")
ax.semilogx(ws, rk, "^-", label="estimated expected risk")
ax.set_xlabel("loss for misclassifying a class-3 sample (w)"); ax.grid(alpha=0.3, which="both")
ax.legend(); ax.set_title("Effect of the class-3 loss weight")
fig.tight_layout(); fig.savefig("figs/q2_sweep.png", dpi=150); plt.close(fig)

json.dump(res, open("out/q2_results.json", "w"), indent=2)
for key in Lambda:
    print(key, res[key]["risk"], res[key]["perr"], res[key]["dec_counts"])
    print(np.round(np.array(res[key]["conf"]), 4))
print("avg std", avg_std, "dists", np.round(dists, 3), res["class_counts"])
```

q3.py

```
"""Question 3: Gaussian class-conditional (QDA-type) minimum-P(error) classifiers on real data."""
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.stats import chi2
from common import gauss_logpdf

from pathlib import Path
from scipy.linalg import solve_triangular
DATA = Path(__file__).resolve().parent / "data"


# ------------------------------------------------------------------ data
def load_wine():
    df = pd.read_csv(f"{DATA}/wine_white.csv", sep=";")
    X = df.drop(columns="quality").to_numpy(float)
    y = df["quality"].to_numpy(int)
    return X, y, list(df.columns[:-1])


def load_har():
    Xtr = np.loadtxt(f"{DATA}/har/X_train.txt"); Xte = np.loadtxt(f"{DATA}/har/X_test.txt")
    ytr = np.loadtxt(f"{DATA}/har/y_train.txt", int); yte = np.loadtxt(f"{DATA}/har/y_test.txt", int)
    names = [l.split()[1] for l in open(f"{DATA}/har/activity_labels.txt")]
    return np.vstack([Xtr, Xte]), np.r_[ytr, yte], names, len(ytr)


# ------------------------------------------------------------------ model
def fit(X, y, alpha, diagnostics=False):
    """Per-class priors (counts), sample-average means and covariances, plus
    lambda_j = alpha * trace(C_j) / rank(C_j) added to the diagonal."""
    model = {}
    for c in np.unique(y):
        Xc = X[y == c]
```

```
        m = Xc.mean(0)
        C = (Xc - m).T @ (Xc - m) / len(Xc) # 1/N_c sample-average estimator
        eig = np.linalg.eigvalsh(C)
        tol = max(C.shape) * np.finfo(float).eps * max(abs(eig))
        r = int(np.sum(eig > tol))
        if r == 0:
            raise ValueError("Zero-rank class covariance requires a separate variance floor")
        lam = alpha * np.trace(C) / r
        model[c] = dict(prior=len(Xc) / len(X), m=m, C=C + lam * np.eye(X.shape[1]), lam=lam,
                        rank=int(r), n=len(Xc), cond_raw=float(np.linalg.cond(C)) if diagnostics else None,
                        cond_reg=float(np.linalg.cond(C + lam * np.eye(X.shape[1]))) if diagnostics else None)
    return model


def predict(model, X):
    cls = np.array(sorted(model))
    S = np.column_stack([gauss_logpdf(X, model[c]["m"], model[c]["C"]) + np.log(model[c]["prior"]) for c in cls])
    return cls[np.argmax(S, 1)]


def confusion(D, y, cls):
    """M[i,j] = P(D=cls_i | L=cls_j), counts as well."""
    K = len(cls)
    cnt = np.zeros((K, K), int)
    for a, ci in enumerate(cls):
        for b, cj in enumerate(cls):
            cnt[a, b] = np.sum((D == ci) & (y == cj))
    return cnt / cnt.sum(0, keepdims=True), cnt


def stratified_folds(y, k, rng):
    f = np.empty(len(y), int)
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        f[idx] = np.arange(len(idx)) % k
    return f


def heat(ax, M, cnt, ticks, title):
    ax.imshow(M, cmap="Blues", vmin=0, vmax=1)
    for i in range(M.shape[0]):
        for j in range(M.shape[1]):
            if cnt[:, j].sum() == 0:
                continue
            ax.text(j, i, f"{M[i, j]:.2f}\n({cnt[i, j]})", ha="center", va="center", fontsize=7,
                    color="white" if M[i, j] > 0.6 else "black")
    ax.set_xticks(range(len(ticks)), [f"L={t}" for t in ticks], fontsize=8, rotation=0)
    ax.set_yticks(range(len(ticks)), [f"D={t}" for t in ticks], fontsize=8)
    ax.set_title(title, fontsize=10)


def pca(X, k):
    Xc = X - X.mean(0)
    U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
    return Xc @ Vt[:k].T, s ** 2 / np.sum(s ** 2)


res = {}
rng = np.random.default_rng(0)
alphas = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 3e-2, 1e-1, 3e-1]

# ================================================================ WINE
Xw_raw, yw, wnames = load_wine()
mu_w, sd_w = Xw_raw.mean(0), Xw_raw.std(0)
Xw = (Xw_raw - mu_w) / sd_w # z-score (see report for why)
cls_w = np.unique(yw)
res["wine_counts"] = {int(c): int((yw == c).sum()) for c in cls_w}

folds = stratified_folds(yw, 5, rng)
sweep_w = []
for a in alphas:
    m = fit(Xw, yw, a)
    tr_err = float(np.mean(predict(m, Xw) != yw))
    cv_err = []
    for f in range(5):
        # Fit preprocessing exclusively on this fold's training portion.
        train, valid = Xw_raw[folds != f], Xw_raw[folds == f]
        center, scale = train.mean(0), train.std(0)
        scale = np.where(scale > 0, scale, 1.0)
        mf = fit((train-center)/scale, yw[folds != f], a)
        cv_err.append(np.mean(predict(mf, (valid-center)/scale) != yw[folds == f]))
    sweep_w.append(dict(alpha=a, train=tr_err, cv=float(np.mean(cv_err))))
res["wine_sweep"] = sweep_w
alpha_w = min(sweep_w, key=lambda r: r["cv"])["alpha"]
```

```
mw = fit(Xw, yw, alpha_w, diagnostics=True)
Dw = predict(mw, Xw)
Mw, Cw = confusion(Dw, yw, cls_w)
res["wine"] = dict(alpha=alpha_w, perr=float(np.mean(Dw != yw)), conf=Mw.tolist(), counts=Cw.tolist(),
                   classes=cls_w.tolist(),
                   per_class={int(c): dict(n=mw[c]["n"], prior=mw[c]["prior"], rank=mw[c]["rank"],
                                           lam=mw[c]["lam"], cond_raw=mw[c]["cond_raw"],
                                           cond_reg=mw[c]["cond_reg"]) for c in cls_w},
                   majority_err=float(1 - max(np.mean(yw == c) for c in cls_w)))
# same classifier on raw (unstandardised) features, for comparison
mw_raw = fit(Xw_raw, yw, alpha_w)
res["wine"]["perr_raw_features"] = float(np.mean(predict(mw_raw, Xw_raw) != yw))
# within-one-grade accuracy
res["wine"]["within1"] = float(np.mean(np.abs(Dw - yw) <= 1))

# ================================================================== HAR
Xh, yh, hnames, ntr = load_har()
mu_h, sd_h = Xh.mean(0), Xh.std(0)
Xhs = (Xh - mu_h) / sd_h
cls_h = np.unique(yh)
res["har_counts"] = {int(c): int((yh == c).sum()) for c in cls_h}
sweep_h = []
for a in alphas:
    m = fit(Xhs, yh, a)
    tr_err = float(np.mean(predict(m, Xhs) != yh))
    # held-out check on the dataset's own subject-disjoint train/test split
    # This official test split is used as validation for alpha, not as an unbiased final test.
    center, scale = Xh[:ntr].mean(0), Xh[:ntr].std(0)
    scale = np.where(scale > 0, scale, 1.0)
    mt = fit((Xh[:ntr]-center)/scale, yh[:ntr], a)
    te_err = float(np.mean(predict(mt, (Xh[ntr:]-center)/scale) != yh[ntr:]))
    sweep_h.append(dict(alpha=a, train=tr_err, test=te_err))
    print("HAR", a, tr_err, te_err, flush=True)
res["har_sweep"] = sweep_h
alpha_h = min(sweep_h, key=lambda r: r["test"])["alpha"]
mh = fit(Xhs, yh, alpha_h, diagnostics=True)
Dh = predict(mh, Xhs)
Mh, Ch = confusion(Dh, yh, cls_h)
res["har"] = dict(alpha=alpha_h, perr=float(np.mean(Dh != yh)), conf=Mh.tolist(), counts=Ch.tolist(),
                  names=hnames,
                  per_class={int(c): dict(n=mh[c]["n"], prior=mh[c]["prior"], rank=mh[c]["rank"],
                                          lam=mh[c]["lam"], cond_raw=mh[c]["cond_raw"],
                                          cond_reg=mh[c]["cond_reg"]) for c in cls_h})
json.dump(res, open("out/q3_results.json", "w"), indent=2, default=float)

# ================================================================== FIGURES
# confusion matrices
fig, ax = plt.subplots(figsize=(6.4, 5.4))
heat(ax, Mw, Cw, cls_w, fr"Wine (white): $P(D=i\mid L=j)$, P(error) = {res['wine']['perr']:.4f}")
fig.tight_layout(); fig.savefig("figs/q3_wine_conf.png", dpi=160); plt.close(fig)
fig, ax = plt.subplots(figsize=(6.4, 5.4))
heat(ax, Mh, Ch, cls_h, fr"HAR: $P(D=i\mid L=j)$, P(error) = {res['har']['perr']:.4f}")
fig.tight_layout(); fig.savefig("figs/q3_har_conf.png", dpi=160); plt.close(fig)

# regularisation sweep
fig, axs = plt.subplots(1, 2, figsize=(11, 3.8))
axs[0].semilogx(alphas, [r["train"] for r in sweep_w], "o-", label="all-sample (training) error")
axs[0].semilogx(alphas, [r["cv"] for r in sweep_w], "s-", label="5-fold CV error")
axs[0].axvline(alpha_w, color="k", ls=":"); axs[0].set_title("Wine"); axs[0].set_xlabel(r"$\alpha$")
axs[1].semilogx(alphas, [r["train"] for r in sweep_h], "o-", label="all-sample (training) error")
axs[1].semilogx(alphas, [r["test"] for r in sweep_h], "s-", label="validation error (official split; used to select alpha)")
axs[1].axvline(alpha_h, color="k", ls=":"); axs[1].set_title("HAR"); axs[1].set_xlabel(r"$\alpha$")
for a in axs:
    a.set_ylabel("P(error)"); a.grid(alpha=0.3, which="both"); a.legend(fontsize=8)
fig.tight_layout(); fig.savefig("figs/q3_alpha_sweep.png", dpi=160); plt.close(fig)

# PCA projections
cmap = plt.get_cmap("tab10")
for nm, X, y, cls, lab in [("wine", Xw, yw, cls_w, lambda c: f"q={c}"),
                           ("har", Xhs, yh, cls_h, lambda c: hnames[c - 1])]:
    Z, evr = pca(X, 3)
    fig = plt.figure(figsize=(13, 4.6))
    ax1 = fig.add_subplot(1, 3, 1); ax2 = fig.add_subplot(1, 3, 2); ax3 = fig.add_subplot(1, 3, 3, projection="3d")
    for k, c in enumerate(cls):
        i = y == c
        kw = dict(s=4, alpha=0.45, color=cmap(k), label=lab(c))
        ax1.scatter(Z[i, 0], Z[i, 1], **kw); ax2.scatter(Z[i, 0], Z[i, 2], **kw)
        ax3.scatter(Z[i, 0], Z[i, 1], Z[i, 2], **kw)
    ax1.set_xlabel(f"PC1 ({evr[0]:.1%})"); ax1.set_ylabel(f"PC2 ({evr[1]:.1%})")
    ax2.set_xlabel("PC1"); ax2.set_ylabel(f"PC3 ({evr[2]:.1%})")
    ax3.set_xlabel("PC1"); ax3.set_ylabel("PC2"); ax3.set_zlabel("PC3")
    ax1.legend(markerscale=4, fontsize=7, loc="best")
    res[nm.lower() + "_pca_explained"] = evr[:3].tolist()
    fig.suptitle(f"{'White wine' if nm == 'wine' else 'HAR'}: first three principal components (standardised features)")
```

```
    fig.tight_layout(); fig.savefig(f"figs/q3_{nm}_pca.png", dpi=150); plt.close(fig)

# wine: feature-subset scatter + marginal histograms
fig, axs = plt.subplots(1, 3, figsize=(13, 4))
pairs = [("alcohol", "density"), ("alcohol", "volatile acidity"), ("residual sugar", "chlorides")]
for ax, (fx, fy) in zip(axs, pairs):
    ix, iy = wnames.index(fx), wnames.index(fy)
    for k, c in enumerate(cls_w):
        i = yw == c
        ax.scatter(Xw_raw[i, ix], Xw_raw[i, iy], s=4, alpha=0.5, color=cmap(k), label=f"q={c}")
    ax.set_xlabel(fx); ax.set_ylabel(fy)
axs[0].legend(markerscale=4, fontsize=7)
fig.suptitle("White wine: raw feature pairs")
fig.tight_layout(); fig.savefig("figs/q3_wine_pairs.png", dpi=150); plt.close(fig)

fig, axs = plt.subplots(2, 6, figsize=(15, 5))
for k, ax in enumerate(axs.ravel()):
    if k >= len(wnames):
        ax.axis("off"); continue
    ax.hist(Xw_raw[yw == 6, k], bins=50, density=True, color="C0", alpha=0.7)
    ax.set_title(wnames[k], fontsize=9); ax.tick_params(labelsize=7)
fig.suptitle("White wine, quality-6 class: marginal histograms (a Gaussian model implies symmetric, bell-shaped marginals)")
fig.tight_layout(); fig.savefig("figs/q3_wine_hist.png", dpi=140); plt.close(fig)

# HAR: a few raw features
fig, axs = plt.subplots(1, 2, figsize=(11, 4.2))
for ax, (fx, fy) in zip(axs, [(0, 1), (40, 52)]): # 0-based column indices into the 561 features
    for k, c in enumerate(cls_h):
        i = yh == c
        ax.scatter(Xh[i, fx], Xh[i, fy], s=3, alpha=0.4, color=cmap(k), label=hnames[c - 1])
    ax.set_xlabel(f"feature {fx + 1}"); ax.set_ylabel(f"feature {fy + 1}")
axs[0].legend(markerscale=4, fontsize=7)
fig.suptitle("HAR: two raw feature pairs")
fig.tight_layout(); fig.savefig("figs/q3_har_pairs.png", dpi=150); plt.close(fig)

# Gaussianity check: squared Mahalanobis distances vs chi^2_d quantiles (Q-Q)
fig, axs = plt.subplots(1, 2, figsize=(11, 4.3))
for ax, (nm, X, y, mdl, cs) in zip(axs, [("Wine", Xw, yw, mw, [5, 6, 7]), ("HAR", Xhs, yh, mh, [1, 4, 6])]):
    d = X.shape[1]
    for c in cs:
        Xc = X[y == c]
        m = Xc.mean(0); C = mdl[c]["C"]
        Lc = np.linalg.cholesky(C)
        r2 = np.sort(np.sum(solve_triangular(Lc, (Xc - m).T, lower=True, check_finite=False) ** 2, 0))
        q = chi2.ppf((np.arange(1, len(r2) + 1) - 0.5) / len(r2), d)
        lab = f"q={c}" if nm == "Wine" else hnames[c - 1]
        ax.plot(q, r2, ".", ms=2, label=lab)
    lim = ax.get_xlim()
    ax.plot(lim, lim, "k--", lw=1, label="y = x (Gaussian)")
    ax.set_xlabel(fr"$\chi^2_{{{d}}}$ quantiles"); ax.set_ylabel("sorted squared Mahalanobis distance")
    ax.set_title(f"{nm}: descriptive Q-Q diagnostic (fitted, regularized)"); ax.legend(markerscale=5, fontsize=8)
    if nm == "Wine":
        ax.set_ylim(0, 150)
    ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig("figs/q3_qq.png", dpi=150); plt.close(fig)

print(json.dumps({k: res[k] for k in ("wine_sweep", "har_sweep")}, indent=1))
print("wine", alpha_w, res["wine"]["perr"], res["wine"]["perr_raw_features"], res["wine"]["within1"], res["wine"]["majority_err"])
print(np.round(Mw, 3)); print(Cw)
print("har", alpha_h, res["har"]["perr"]); print(np.round(Mh, 4)); print(Ch)
for c in cls_w: print(c, res["wine"]["per_class"][int(c)])
for c in cls_h: print(c, res["har"]["per_class"][int(c)])

json.dump(res, open("out/q3_results.json", "w"), indent=2, default=float)
```